UNIVERSIDAD DE ALICANTE

Sistemas Distribuidos · Curso académico 2026/2027

Water Management Network

Práctica no guiada: Sockets, Streaming de Eventos, Colas y modularidad

Memoria de prácticas · Release 1 (estado de avance)

Autores

Laura Caturla Martín DNI: _____

Borja Lara _____ DNI: _____

Versión del documento: 11 de octubre de 2026 · Recoge el trabajo realizado hasta la fecha sobre la comunicación por sockets entre WM_Central, WM_WS_M y WM_WS_E.

Índice

1. Introducción y estado de avance
2. Arquitectura del sistema
3. Protocolo de comunicación por sockets
4. Estados de una estación de riego
5. Componentes software desarrollados
6. Concurrencia y tolerancia a fallos
7. Guía de despliegue
8. Pruebas y capturas de funcionamiento
9. Trabajo pendiente
10. Uso de herramientas de IA
11. Anexo: código fuente

1. Introducción y estado de avance

WaterManagement es un sistema distribuido que simula la gestión centralizada de una red de estaciones de riego (*Watering Stations*, WS) repartidas por los parques y jardines de una ciudad. Una central (`WM_Central`) da de alta las estaciones, guarda su información en una base de datos SQLite y muestra su estado en tiempo real en un panel de monitorización. Cada estación está formada por dos procesos: un **Monitor** (`WM_WS_M`), que se autentica en la central y vigila la salud de la estación, y un **Engine** (`WM_WS_E`), que simula el hardware (caudalímetro, detector de fugas y electroválvula).

Este documento describe el trabajo realizado hasta la fecha. Se ha completado toda la parte del sistema basada en **sockets TCP**: registro y autenticación de estaciones, vigilancia de salud, detección de fugas, detección de desconexiones y persistencia del estado. La parte basada en **streaming de eventos con Kafka** (peticiones de riego, telemetría y aplicación del operario) es la siguiente fase del desarrollo.

Funcionalidad	Componente	Estado
Servidor de sockets concurrente (un hilo por estación)	WM_Central	**Implementado**
Registro y autenticación de estaciones (rechazo de ID duplicado)	WM_Central, WM_WS_M	**Implementado**
Persistencia en SQLite y recuperación tras reinicio	WM_Central	**Implementado**
Panel de monitorización con colores por estado	WM_Central	**Implementado**
Vigilancia de salud cada segundo (HEALTH / OK / KO)	WM_WS_M, WM_WS_E	**Implementado**
Simulación de averías por teclado y reconexión automática	WM_WS_E	**Implementado**
Detección de desconexión de Monitor o Engine	WM_Central, WM_WS_M	**Implementado**
Peticiones de riego, telemetría y notificaciones (Kafka)	Todos, WM_FO	**Pendiente**
Órdenes de Central: iniciar riego, bloquear, activar	WM_Central, WM_WS_E	**Pendiente**
Tabla de operarios en la base de datos	WM_Central	**Pendiente**
Despliegue con Docker y Railway	Todos	**Pendiente**

2. Arquitectura del sistema

La figura 1 muestra los componentes y cómo se comunican. En trazo continuo aparece lo implementado y en trazo discontinuo lo que se añadirá con Kafka. La comunicación entre la central y cada estación sigue una cadena de dos sockets TCP: la central es servidor del Monitor, y el Monitor es a su vez servidor del Engine de su misma estación.

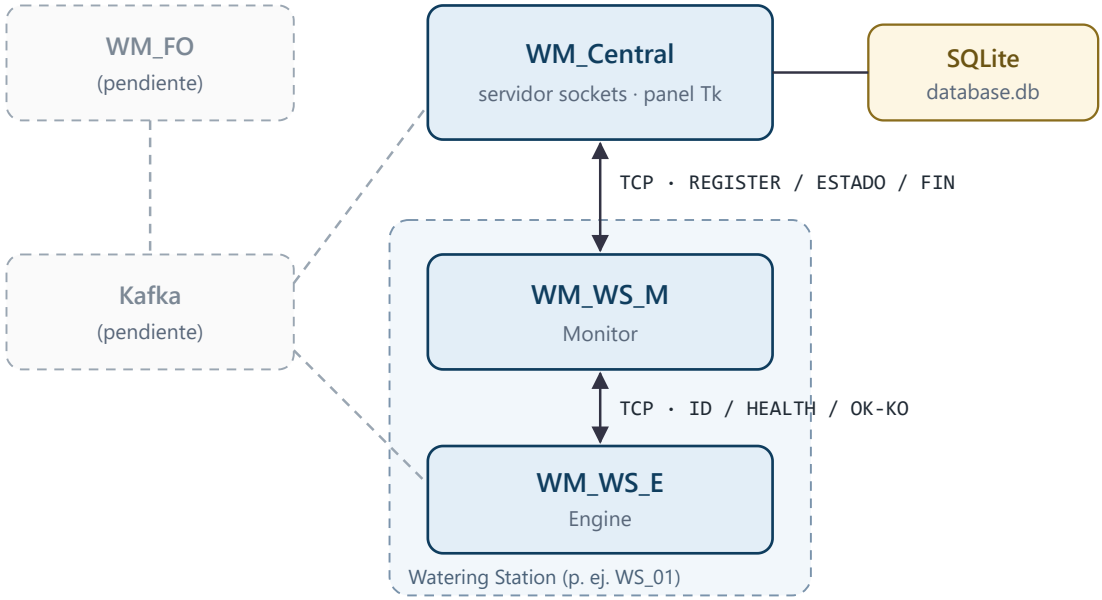


Figura 1. Arquitectura actual. Componentes implementados (trazo continuo) y pendientes (discontinuo).

La tabla siguiente resume los hilos de ejecución de cada proceso. Usar varios hilos permite que la central atienda a muchas estaciones a la vez y que Monitor y Engine sigan funcionando mientras esperan órdenes por teclado.

Proceso	Hilo	Responsabilidad
WM_Central	Principal	Panel Tkinter. Relee la base de datos cada 500 ms y pinta cada fila con el color de su estado.
	`start` (daemon)	Acepta conexiones en el puerto de escucha y crea un hilo por cada una.
	`handle_client` (uno por Monitor)	Recibe los mensajes de una estación, actualiza memoria y BD, y responde con ACK/NACK.
WM_WS_M	Principal	Menú por consola (comando `FIN` para desconectarse).
	`servidor_engine` (daemon)	Espera la conexión del Engine, le pregunta su salud cada segundo y notifica los cambios de estado a la central.
WM_WS_E	Principal	Lee el teclado para simular averías (`K`), resolverlas (`O`) o salir (`FIN`).
	`atender_monitor` (daemon)	Se conecta al Monitor, responde a cada `HEALTH` y se reconecta si el Monitor cae.

3. Protocolo de comunicación por sockets

3.1 Formato de trama

TCP transmite un flujo continuo de bytes, sin separación entre mensajes, por lo que hay que delimitarlos. Todos los componentes usan el mismo formato:

```
[ CABECERA: 64 bytes ][ MENSAJE: N bytes en UTF-8 ]
  "23" + 62 espacios     "REGISTER#WS_01#Parque"
```

- La **cabecera** ocupa siempre 64 bytes y contiene la longitud del mensaje en texto, rellenada con espacios.
- El **mensaje** lleva el código de operación y los campos separados por `#`: `CODIGO#campo1#...#campoN`.
- Para recibir, primero se leen exactamente 64 bytes, se obtiene N y después se leen exactamente N bytes. Como `recv` puede devolver menos bytes de los pedidos, la función `recive_msg` repite la lectura hasta completarlos. Si devuelve 0 bytes, la conexión se ha cerrado y la función devuelve `None`.
- Para enviar se usa `sendall`, que garantiza que se transmite la trama completa.

> El enunciado recomienda (sin obligar) el empaquetado `<STX><DATA><ETX><LRC>`. Se ha optado por una cabecera de longitud fija, que resuelve igualmente la delimitación de mensajes. La verificación de integridad mediante LRC queda como posible mejora.

3.2 Mensajes Monitor ↔ Central

Petición (WM_WS_M → WM_Central)	Respuesta	Significado
`REGISTER#id#ubicacion`	`ACK#REGISTER_OK#id` `NACK#REGISTER_KO#id#motivo`	Alta y autenticación. Se rechaza si el ID ya tiene un Monitor conectado.
`ESTADO#id#estado`	`ACK#ESTADO_OK#id#estado` `NACK#ESTADO_KO#id`	Cambio de estado. Solo se acepta para la estación de esa conexión y con un estado válido.
`FIN`	(sin respuesta)	Desconexión ordenada. La estación pasa a `OFFLINE`.
Otro texto	`ACK#OK#Recibido: ...`	Eco de depuración desde el menú del Monitor.

3.3 Mensajes Monitor ↔ Engine

Mensaje	Sentido	Significado
`ID#id`	Monitor → Engine	Primer mensaje tras la conexión. Asigna al Engine el ID ya validado por la central.
`HEALTH`	Monitor → Engine	Comprobación de salud, enviada cada segundo.
`OK` / `KO`	Engine → Monitor	Funcionamiento correcto, o avería (fuga) simulada. Si no llega respuesta en 3 s, el Monitor lo trata como avería.

3.4 Ejemplo de intercambio completo

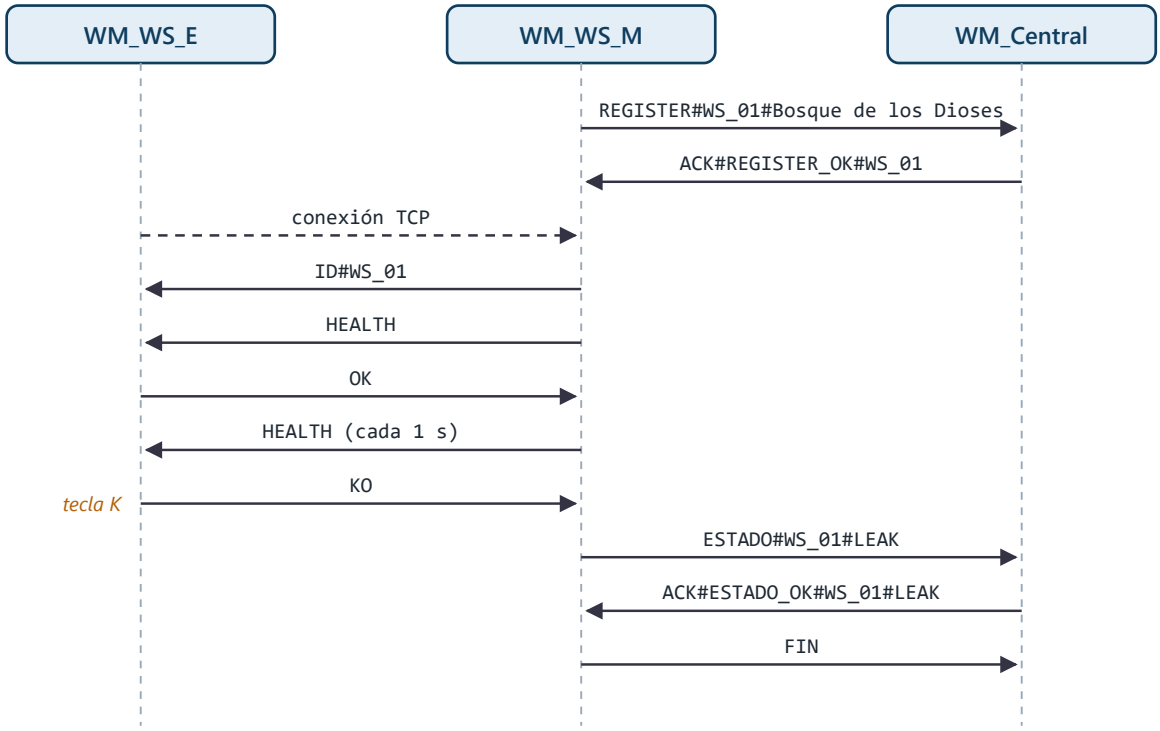


Figura 2. Registro de WS_01, vigilancia de salud, detección de una fuga y desconexión.

4. Estados de una estación de riego

Los estados se guardan en la columna `Status` de la base de datos con un código en inglés. El panel de la central los muestra con el texto y el color que fija el enunciado.

Código	Panel	Significado	Quién lo provoca
`AVAILABLE`	🟩 Disponible	Funciona correctamente y espera una petición de riego.	Registro aceptado, o fin de una fuga **(impl.)**
`WATERING`	🟩 Regando (parpadea)	Está regando. Mostrará caudal, volumen y operario.	Orden de riego vía Kafka **(pend.)**
`LEAK`	🟥 Fuga	Conectada pero averiada. No debe regar.	Monitor: Engine responde KO o deja de responder **(impl.)**
`STOPPED`	🟧 Fuera de Servicio	Funciona, pero bloqueada por orden de la central.	Orden de bloqueo de la central **(pend.)**
`OFFLINE`	⬜ Desconectada	La central no tiene conexión con su Monitor.	Central: FIN, caída del socket o reinicio **(impl.)**

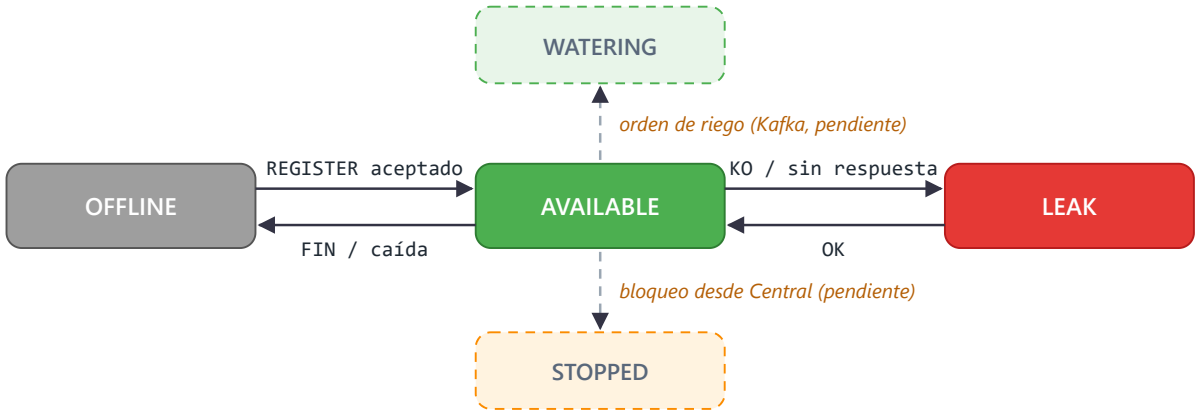


Figura 3. Transiciones implementadas (continuas) y previstas para la fase de Kafka (discontinuas). Desde cualquier estado se pasa a OFFLINE si se pierde la conexión con el Monitor.

El Monitor solo notifica a la central las **transiciones**: si el estado no cambia, no envía nada. Así se evita mandar un mensaje cada segundo mientras todo funciona con normalidad.

5. Componentes software desarrollados

5.1 WM_Central

```
python WM_Central.py <puerto_escucha>
```

Es el núcleo del sistema. Al arrancar realiza estos pasos:

1. **Recuperación desde la BD** (`cargar_estaciones_bd`). Lee las estaciones ya registradas y las marca como `OFFLINE`, ya que hasta que su Monitor no conecte no se conoce su estado real (punto 1 de la mecánica). También borra los datos de riego anteriores (`Flow`, `Accum`, `Operator`).
2. **Servidor de sockets**. Escucha en `0.0.0.0:<puerto>` en un hilo aparte y crea un hilo `handle_client` por cada Monitor que se conecta.
3. **Panel de monitorización**. Ventana Tkinter con una tabla (ID, ubicación, estado, caudal, volumen acumulado y operario) y el mapa de zonas. Se refresca cada 500 ms. Cada estado tiene un *tag* con su color de fondo, y las estaciones en `WATERING` alternan entre dos tonos de verde para parpadear.

Autenticación. La función `registrar_estacion` rechaza con `NACK` un ID que ya tiene un Monitor conectado y cierra esa conexión. Una estación rechazada no queda asociada a la conexión, de modo que su cierre no afecta a la estación legítima. Además, un Monitor solo puede cambiar el estado de su propia estación.

Desconexión. Cuando un Monitor envía `FIN` o su conexión se corta (cierre del proceso, fallo de red, `ConnectionResetError`), la estación pasa a `OFFLINE` en memoria y en la base de datos.

5.2 Base de datos

Se usa SQLite (fichero `database.db`), como exige el enunciado. Cada operación abre su propia conexión, lo que permite usarla desde varios hilos sin compartir el objeto de conexión.

Columna	Tipo	Contenido
`ID`	texto (clave)	Identificador único de la estación.
`Location`	texto	Parque, jardín o zona de la ciudad.
`Status`	texto	Estado actual (ver sección 4).
`Flow`	real	Caudal instantáneo en l/min (se usará con Kafka).
`Accum`	entero	Volumen acumulado del riego en litros (se usará con Kafka).
`Operator`	texto	Operario que ha activado el riego (se usará con Kafka).

5.3 WM_WS_M (Monitor)

```
python WM_WS_M.py <puerto_engine> <ip_central> <puerto_central> <id_ws> <ubicacion>
```

1. Se conecta a la central y envía `REGISTER#id#ubicacion`. Si la respuesta no es `ACK#REGISTER_OK`, termina.

2. Arranca el hilo `servidor_engine`, que abre un servidor en `puerto_engine` y espera al Engine. Al conectarse este, le envía `ID#id`.
3. La función `vigilar_engine` envía `HEALTH` cada segundo con un tiempo de espera de 3 s:
   - `OK`: la estación queda en `AVAILABLE`.
   - `KO` u otra respuesta: pasa a `LEAK` y sigue preguntando. Cuando vuelva `OK`, la avería se da por resuelta.
   - Sin respuesta o conexión cortada: pasa a `LEAK` y espera a que el Engine se reconecte.
4. `cambiar_estado` envía `ESTADO#id#estado` a la central solo cuando el estado cambia.
5. El hilo principal mantiene el menú. `FIN` cierra la conexión con la central.

5.4 WM_WS_E (Engine)

```
python WM_WS_E.py <ip_broker> <puerto_broker> <ip_monitor> <puerto_monitor>
```

Simula el hardware de la estación. El hilo `atender_monitor` se conecta al Monitor, guarda el ID recibido y responde a cada `HEALTH` con el valor actual de `estado_engine`. Si el Monitor no está disponible o cae, reintenta la conexión cada 3 segundos. El hilo principal lee el teclado para que se puedan provocar incidencias en tiempo de ejecución, como pide el enunciado:

Comando	Efecto
`K`	Simula una fuga: el Engine responde `KO` al Monitor.
`O`	Resuelve la fuga: el Engine vuelve a responder `OK`.
`FIN`	Apaga el Engine. El Monitor lo detecta y notifica `LEAK`.

La lectura se hace con `input()` (comando y Enter) en lugar de capturar una sola tecla, porque las librerías de lectura de teclas sueltas dependen del sistema operativo y el despliegue final será en contenedores Docker (Linux). Los parámetros del broker ya se reciben, aunque no se usarán hasta incorporar Kafka.

6. Concurrencia y tolerancia a fallos

6.1 Secciones críticas

Recurso	Protección	Problema que evita
Socket Monitor → Central	`lock_central`	El menú y el hilo de vigilancia comparten el socket. Sin el lock, un hilo podría enviar su petición y quedarse con la respuesta que correspondía al otro. El lock hace que cada `send` y su `receive` sean una operación atómica (`hablar_central`).
Diccionario `estaciones` de la central	`lock_estaciones`	Cada Monitor se atiende en un hilo distinto. La comprobación de ID duplicado y el alta se hacen bajo el mismo lock, para que dos Monitores con el mismo ID que lleguen a la vez no puedan registrarse ambos.
`estado_engine`	Sin lock	Solo lo escribe el hilo del teclado y solo lo lee el del Monitor. Es una asignación simple, que no puede quedar a medias.

6.2 Situaciones de fallo contempladas

Situación	Comportamiento del sistema
El Engine responde `KO`	El Monitor notifica `LEAK`. Al volver `OK`, notifica `AVAILABLE`.
El Engine se cae o no responde en 3 s	El Monitor notifica `LEAK` y espera su reconexión. El Engine reintenta conectarse cada 3 s.
El Monitor se cae (sin enviar FIN)	La central detecta el cierre del socket y marca la estación como `OFFLINE`.
La central se reinicia	Conserva las estaciones en la BD y las muestra como `OFFLINE` hasta que vuelvan a registrarse.
Se registra un ID ya conectado	La central responde `NACK#REGISTER_KO` y el Monitor intruso termina. La estación original no se ve afectada.
`recv` devuelve un mensaje incompleto	`recive_msg` sigue leyendo hasta completar la longitud indicada en la cabecera.
Cabecera corrupta	La central captura el error y cierra solo esa conexión, sin afectar al resto.

7. Guía de despliegue

Esta es la guía para la versión actual, ejecutada en local. Solo requiere Python 3 (probado con 3.12) y su biblioteca estándar (`socket`, `threading`, `sqlite3`, `tkinter`).

> El despliegue definitivo exigido por el enunciado (Docker en todos los módulos; WM_Central, BD y Kafka en Railway; WM_FO y WM_WS en los PCs del laboratorio) se documentará en la siguiente versión de esta memoria.

Paso 1. Central

Debe ejecutarse desde la carpeta `Central`, ya que abre `database.db` e `InvernaliaZonas.png` con ruta relativa.

```
cd Central
python WM_Central.py 9999
```

Paso 2. Monitor de cada estación

```
python WM_WS_M.py 8000 <ip_central> 9999 WS_01 "Bosque de los Dioses"
python WM_WS_M.py 8001 <ip_central> 9999 WS_02 "Patio de Armas"
```

Paso 3. Engine de cada estación

Cada Engine se conecta al puerto de su Monitor. La IP y el puerto del broker (9092) se reservan para Kafka.

```
python WM_WS_E.py <ip_broker> 9092 <ip_monitor> 8000
python WM_WS_E.py <ip_broker> 9092 <ip_monitor> 8001
```

Paso 4. Comprobación

1. Las dos estaciones aparecen en verde (*Disponible*) en el panel.
2. Escribir `K` en el Engine de WS_01: la fila pasa a rojo (*Fuga*). Con `O` vuelve a verde.
3. Escribir `FIN` en el Monitor de WS_02: la fila pasa a gris (*Desconectada*).
4. Cerrar y volver a abrir la central: las estaciones siguen en la tabla, en gris.

Si los procesos están en máquinas distintas, los puertos de escucha de la central (9999) y de cada Monitor (8000, 8001…) deben estar permitidos en el cortafuegos.

8. Pruebas y capturas de funcionamiento

Escenario probado: una central y dos estaciones (WS_01 y WS_02), cada una con su Monitor y su Engine, siguiendo los pasos de la guía de despliegue. Las capturas y salidas de consola corresponden a esa ejecución.

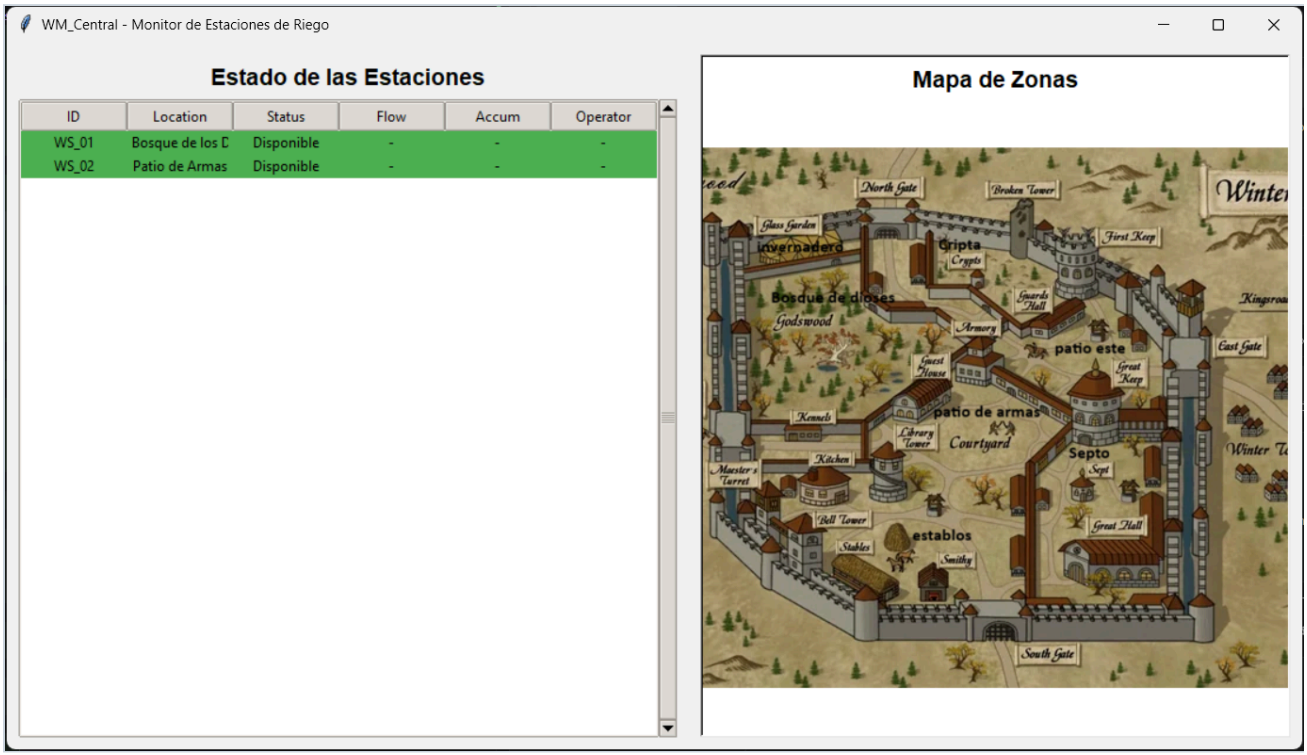


Figura 4. Panel de la central con las dos estaciones registradas y disponibles.



Figura 5. Tras pulsar K en el Engine de WS_01, el Monitor notifica la fuga y la central la muestra en rojo.

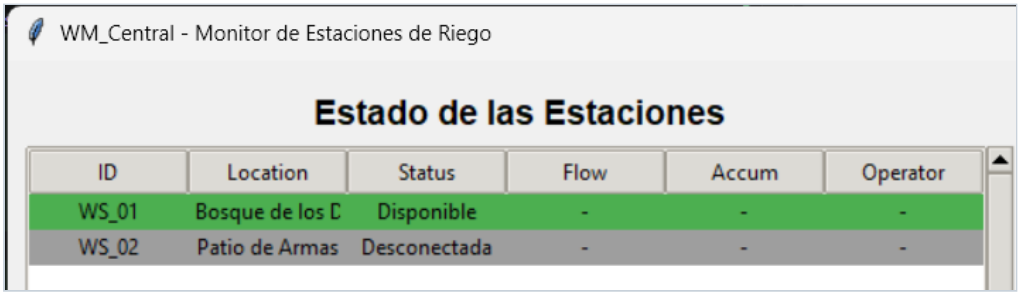


Figura 6. Fuga resuelta con O (WS_01 vuelve a verde) y WS_02 desconectada con FIN (gris).



Figura 7. Tras reiniciar la central, las estaciones se recuperan de la BD en estado Desconectada.

Consola de WM_WS_E (WS_01)

```
****** WM_WS_E - ENGINE DE ESTACIÓN DE RIEGO ****
Comandos: K = provocar avería (KO) | O = resolver avería (OK) | FIN = salir

[MONITOR] Conectado al monitor en ('127.0.0.1', 8000)
[MONITOR] ID asignado a este engine: WS_01
[AVERÍA] Fuga simulada: se responderá KO al monitor
[AVERÍA] Fuga resuelta: se responderá OK al monitor
[DESCONEXIÓN] Apagando engine.
```

Consola de WM_WS_M (WS_01)

```
****** WELCOME TO OUR BRILLIANT SD UA CURSO 2026/2027 SOCKET CLIENT ****
Conexión establecida con ('127.0.0.1', 9999)
[RESPUESTA CENTRAL]: ACK#REGISTER_OK#WS_01
Introduce comando/estado o 'FIN' para salir: [ENGINE] Esperando conexión del engine en el puerto 8000

[ENGINE] Engine conectado desde ('127.0.0.1', 49769)

[ENGINE] Respuesta de avería recibida: KO

[ESTADO] WS_01 pasa a LEAK
[RESPUESTA CENTRAL]: ACK#ESTADO_OK#WS_01#LEAK

[ENGINE] Respuesta de avería recibida: KO

[ENGINE] Respuesta de avería recibida: KO

[ENGINE] Respuesta de avería recibida: KO

[ESTADO] WS_01 pasa a AVAILABLE
[RESPUESTA CENTRAL]: ACK#ESTADO_OK#WS_01#AVAILABLE

[ENGINE] El engine no responde o se ha desconectado

[ESTADO] WS_01 pasa a LEAK
[RESPUESTA CENTRAL]: ACK#ESTADO_OK#WS_01#LEAK
[ENGINE] Esperando reconexión del engine en el puerto 8000
[DESCONEXIÓN] Cerrando conexión.
```

Intento de registro con un ID ya conectado

```
****** WELCOME TO OUR BRILLIANT SD UA CURSO 2026/2027 SOCKET CLIENT ****
Conexión establecida con ('127.0.0.1', 9999)
Registro rechazado o conexión cerrada por la central: NACK#REGISTER_KO#WS_01#ID ya conectado
```

Consola de WM_Central

```
[BD] 0 estaciones cargadas de la base de datos (todas en OFFLINE)
[STARTING] Iniciando WM_Central en 0.0.0.0:9999...
[LISTENING] Servidor a la escucha en 0.0.0.0
[NUEVA CONEXION] ('127.0.0.1', 55458) connected.[CONEXIONES ACTIVAS] 1


[REGISTRO OK] Estación 'WS_01' (Bosque de los Dioses) registrada con éxito desde ('127.0.0.1', 55458)
('WS_01', 'Bosque de los Dioses', 'AVAILABLE', None, None, None)
[NUEVA CONEXION] ('127.0.0.1', 49768) connected.[CONEXIONES ACTIVAS] 2


[REGISTRO OK] Estación 'WS_02' (Patio de Armas) registrada con éxito desde ('127.0.0.1', 49768)
('WS_01', 'Bosque de los Dioses', 'AVAILABLE', None, None, None)
('WS_02', 'Patio de Armas', 'AVAILABLE', None, None, None)
[NUEVA CONEXION] ('127.0.0.1', 49774) connected.

[REGISTRO KO] 'WS_01' ya está conectada, se rechaza ('127.0.0.1', 49774)
[CONEXIONES ACTIVAS] 3
[ESTADO] Estación 'WS_01' -> LEAK
('WS_01', 'Bosque de los Dioses', 'LEAK', None, None, None)
('WS_02', 'Patio de Armas', 'AVAILABLE', None, None, None)
[ESTADO] Estación 'WS_01' -> AVAILABLE
('WS_01', 'Bosque de los Dioses', 'AVAILABLE', None, None, None)
('WS_02', 'Patio de Armas', 'AVAILABLE', None, None, None)
[FIN] La estación 'WS_02' ha enviado FIN
[DESCONEXION] Estación 'WS_02' marcada como OFFLINE
('WS_01', 'Bosque de los Dioses', 'AVAILABLE', None, None, None)
('WS_02', 'Patio de Armas', 'OFFLINE', None, None, None)
[ESTADO] Estación 'WS_01' -> LEAK
('WS_01', 'Bosque de los Dioses', 'LEAK', None, None, None)
('WS_02', 'Patio de Armas', 'OFFLINE', None, None, None)
[FIN] La estación 'WS_01' ha enviado FIN
[DESCONEXION] Estación 'WS_01' marcada como OFFLINE
('WS_01', 'Bosque de los Dioses', 'OFFLINE', None, None, None)
('WS_02', 'Patio de Armas', 'OFFLINE', None, None, None)
```

Consola de WM_Central tras el reinicio

```
[BD] 2 estaciones cargadas de la base de datos (todas en OFFLINE)
('WS_01', 'Bosque de los Dioses', 'OFFLINE', None, None, None)
('WS_02', 'Patio de Armas', 'OFFLINE', None, None, None)
[STARTING] Iniciando WM_Central en 0.0.0.0:9999...
[LISTENING] Servidor a la escucha en 0.0.0.0
```

9. Trabajo pendiente

1. **Tabla de operarios** en la BD y columnas para el riego en curso (hora de inicio y límite de tiempo).
2. **Kafka**: levantar el broker con Docker y definir los topics. Propuesta inicial:
   - `peticiones_riego`: WM_FO → WM_Central.
   - `ordenes_ws`: WM_Central → WM_WS_E (regar, parar, bloquear, activar).
   - `telemetria`: WM_WS_E → WM_Central, cada segundo (caudal y volumen acumulado).
   - `notificaciones_fo`: WM_Central → WM_FO (autorización, denegación y resumen final).
3. **WM_FO**: petición manual y lectura de peticiones desde fichero, con 4 s de espera entre servicios.
4. **Lógica de riego** en el Engine: apertura de la electroválvula, envío de telemetría, fin por tiempo o desde el menú, y corte inmediato si se detecta una fuga durante el riego.
5. **Órdenes arbitrarias de la central** (punto 11): iniciar riego, bloquear (`STOPPED`) y activar, sobre una o todas las estaciones.
6. **Argumentos del broker** en WM_Central.
7. **Despliegue**: Dockerfiles de todos los módulos, Railway para la parte cloud y ciclo CI/CD desde GitHub.

10. Uso de herramientas de IA

Durante el desarrollo se ha utilizado un asistente de IA (Claude) como apoyo para analizar el enunciado, revisar el código existente, proponer la implementación de los estados del Monitor, del Engine y de las mejoras de la central, y redactar esta memoria. El grupo asume la responsabilidad de comprender, explicar y modificar todo el código entregado. Como exige el enunciado, los *prompts* utilizados se incluirán en el repositorio de GitHub y se compartirán con el profesorado.

Anexo: código fuente

A.1 Central/WM_Central.py

```
import socket
import threading
import sys
import sqlite3
import tkinter as tk
from tkinter import ttk


HEADER = 64
FORMAT = 'utf-8'
FIN = "FIN"

# Disponible, Regando, Fuga, Fuera de servicio, Desconectada
ESTADOS_WS = ("AVAILABLE", "WATERING", "LEAK", "STOPPED", "OFFLINE")

# Texto y color con que se muestra cada estado en el panel
TEXTO_ESTADO = {
    "AVAILABLE": "Disponible",
    "WATERING": "Regando",
    "LEAK": "Fuga",
    "STOPPED": "Fuera de Servicio",
    "OFFLINE": "Desconectada"
}
COLOR_ESTADO = {
    "AVAILABLE": "#4caf50",     # verde
    "WATERING": "#4caf50",      # verde (parpadea alternando con WATERING_OFF)
    "WATERING_OFF": "#c8e6c9",  # verde claro
    "LEAK": "#e53935",          # rojo
    "STOPPED": "#fb8c00",       # naranja
    "OFFLINE": "#9e9e9e"        # gris
}

conexion = sqlite3.connect("database.db")
cursor = conexion.cursor()

#Creación de todas las tablas
cursor.execute("""
    CREATE TABLE IF NOT EXISTS estaciones(
        ID TEXT PRIMARY KEY,
        Location TEXT NOT NULL,
        Status TEXT NOT NULL,
        Flow REAL,
        Accum INTERGER,
        Operator TEXT
    )
""")

#Guardar
conexion.commit()

# Diccionario en memoria con todas las estaciones. Lo usan varios hilos a la vez
# (uno por cada monitor conectado), así que se protege con un lock.
estaciones = {}
lock_estaciones = threading.Lock()

def cargar_estaciones_bd():
    # Al arrancar no sabemos el estado real de ninguna estación hasta que se
    # conecte, así que todas las registradas pasan a OFFLINE (punto 1 de la mecánica)
    conexion = sqlite3.connect("database.db")
    cursor = conexion.cursor()

    cursor.execute("""
```

```
        UPDATE estaciones
        SET Status = 'OFFLINE', Flow = NULL, Accum = NULL, Operator = NULL
    """)
    conexion.commit()

    cursor.execute("SELECT ID, Location FROM estaciones")
    for id_estacion, ubicacion in cursor.fetchall():
        estaciones[id_estacion] = {
            "id": id_estacion,
            "ubicacion": ubicacion,
            "estado": "OFFLINE",
            "addr": None,
            "conn": None
        }

    conexion.close()

def guardar_estacion_bd(estaciones, id):
    conexion = sqlite3.connect("database.db")
    cursor = conexion.cursor()

    estacion = estaciones[id]

    id_estacion = estacion["id"]
    ubicacion = estacion ["ubicacion"]
    estado = estacion["estado"]

    cursor.execute("SELECT * FROM estaciones WHERE ID = ?", (id_estacion,))

    if cursor.fetchone() is not None:
        cursor.execute("""
            UPDATE estaciones
            SET Status = ?, Location = ?
            WHERE ID = ?
        """, (estado, ubicacion, id_estacion))
    else:
        cursor.execute("""
            INSERT INTO estaciones (ID, Location, Status)
            VALUES (?,?,?)
            """, (id_estacion,ubicacion, estado)
        )
    conexion.commit()
    conexion.close()

def mostrar_bd():
    conexion = sqlite3.connect("database.db")
    cursor = conexion.cursor()

    cursor.execute("SELECT * FROM estaciones")

    estaciones_bd = cursor.fetchall()

    for estacion in estaciones_bd:
        print(estacion)

    conexion.close()

def modificar_estado(id, estado):
    conexion = sqlite3.connect("database.db")
    cursor = conexion.cursor()

    cursor.execute("""
        UPDATE estaciones
        SET Status = ?
        WHERE ID = ?
    """, (estado, id))

    conexion.commit()
    conexion.close()
```

```
132
133 def registrar_estacion(id_estacion, ubicacion, conn, addr):
134     # Autenticación: se rechaza un ID que ya tiene un monitor conectado.
135     # Comprobar y dar de alta va dentro del lock para que dos monitores con el
136     # mismo ID que lleguen a la vez no puedan pasar los dos.
137     with lock_estaciones:
138         if id_estacion in estaciones and estaciones[id_estacion]["conn"] is not None:
139             return False
140
141         estaciones[id_estacion] = {
142             "id": id_estacion,
143             "ubicacion": ubicacion,
144             "estado": "AVAILABLE",
145             "addr": addr,
146             "conn": conn
147         }
148         guardar_estacion_bd(estaciones, id_estacion)
149     return True
150
151 def desconectar_estacion(id_estacion):
152     with lock_estaciones:
153         estaciones[id_estacion]["estado"] = "OFFLINE"
154         estaciones[id_estacion]["conn"] = None
155         estaciones[id_estacion]["addr"] = None
156         modificar_estado(id_estacion, "OFFLINE")
157
158 def send_msg(conn, msg):
159     message = msg.encode(FORMAT)
160     header = f"{len(message):<{HEADER}}".encode(FORMAT)
161     conn.sendall(header + message)
162
163 def recive_msg(sock, size):
164     # recv puede devolver menos bytes de los pedidos: se repite hasta tenerlos todos
165     data = bytearray()
166     while len(data) < size:
167         part = sock.recv(size - len(data))
168         if not part:
169             return None
170         data.extend(part)
171     return bytes(data)
172
173 def receive_msg(conn):
174     header = recive_msg(conn, HEADER)
175     if header is None:
176         return None
177
178     msg_length = int(header.decode(FORMAT).strip())
179     message = recive_msg(conn, msg_length)
180     if message is None:
181         return None
182
183     return message.decode(FORMAT)
184
185 def handle_client(conn, addr):
186     print(f"[NUEVA CONEXION] {addr} connected.")
187
188     id_estacion_actual = None
189     connected = True
190     while connected:
191             try:
192                 msg = receive_msg(conn)
193             except (OSError, ValueError):  # conexión cortada o cabecera corrupta
194                 msg = None
195
196             if msg is None:
197                 connected = False
198             elif msg == FIN:
199                 connected = False
200                 print(f"[FIN] La estación '{id_estacion_actual}' ha enviado FIN")
```

```
            else:
                partes = msg.split('#')
                comando = partes[0]

                if comando == "REGISTER" and len(partes) >= 3 and id_estacion_actual is None:
                    id_estacion = partes[1]
                    ubicacion = partes[2]

                    if registrar_estacion(id_estacion, ubicacion, conn, addr):
                        id_estacion_actual = id_estacion
                        print(f"\n[REGISTRO OK] Estación '{id_estacion}' ({ubicacion}) registrada con éxito d
esde {addr}")
                        send_msg(conn, f"ACK#REGISTER_OK#{id_estacion}")
                        mostrar_bd()
                    else:
                        print(f"\n[REGISTRO KO] '{id_estacion}' ya está conectada, se rechaza {addr}")
                        send_msg(conn, f"NACK#REGISTER_KO#{id_estacion}#ID ya conectado")
                        connected = False

                elif comando == "ESTADO" and len(partes) >= 3:
                    id_estacion = partes[1]
                    nuevo_estado = partes[2]

                    # Un monitor solo puede cambiar el estado de su propia estación
                    if id_estacion == id_estacion_actual and nuevo_estado in ESTADOS_WS:
                        with lock_estaciones:
                            estaciones[id_estacion]["estado"] = nuevo_estado
                            modificar_estado(id_estacion, nuevo_estado)
                        print(f"[ESTADO] Estación '{id_estacion}' -> {nuevo_estado}")
                        send_msg(conn, f"ACK#ESTADO_OK#{id_estacion}#{nuevo_estado}")
                        mostrar_bd()
                    else:
                        print(f"[ESTADO] Mensaje de estado no válido: {msg}")
                        send_msg(conn, f"NACK#ESTADO_KO#{id_estacion}")

                else:
                    print(f"[CENTRAL] Recibido de [{id_estacion_actual or addr}]: {msg}")
                    respuesta = f"ACK#OK#Recibido: {msg}"
                    send_msg(conn, respuesta)

    # Tanto si llega FIN como si se corta la conexión, la estación queda desconectada
    if id_estacion_actual is not None:
        desconectar_estacion(id_estacion_actual)
        print(f"[DESCONEXION] Estación '{id_estacion_actual}' marcada como OFFLINE")
        mostrar_bd()

    conn.close()



def start(server, ip, port):
    server.listen()
    print(f"[LISTENING] Servidor a la escucha en {SERVER}")
    while True:
        conn, addr = server.accept()
        thread = threading.Thread(target=handle_client, args=(conn, addr), daemon=True)
        thread.start()
        print(f"[CONEXIONES ACTIVAS] {threading.active_count() - 2}")

def Pantalla_Monitor():
    ventana = tk.Tk()
    ventana.title("WM_Central - Monitor de Estaciones de Riego")
    ventana.geometry("1100x600")

    # El tema por defecto de Windows ignora el color de fondo de las filas; "clam" sí lo respeta
    estilo = ttk.Style()
    estilo.theme_use("clam")

    # 1. Frame Izquierdo: Tabla de Datos
```

```
    frame_izq = tk.Frame(ventana)
    frame_izq.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=10, pady=10)

    lbl_tabla = tk.Label(frame_izq, text="Estado de las Estaciones", font=("Arial", 14, "bold"))
    lbl_tabla.pack(pady=5)

    columnas = ("ID", "Location", "Status", "Flow", "Accum", "Operator")
    tabla = ttk.Treeview(frame_izq, columns=columnas, show="headings")

    for col in columnas:
        tabla.heading(col, text=col)
        tabla.column(col, width=90, anchor=tk.CENTER)

    # Una etiqueta (tag) por estado con su color de fondo
    for estado, color in COLOR_ESTADO.items():
        tabla.tag_configure(estado, background=color)

    scrollbar = ttk.Scrollbar(frame_izq, orient=tk.VERTICAL, command=tabla.yview)
    tabla.configure(yscroll=scrollbar.set)
    scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
    tabla.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

    # Alterna en cada refresco para que las estaciones regando parpadeen
    parpadeo = {"encendido": True}

    # Función interna para refrescar los datos automáticamente
    def cargar_datos_bd():
        for fila in tabla.get_children():
            tabla.delete(fila)

        parpadeo["encendido"] = not parpadeo["encendido"]

        try:
            conn = sqlite3.connect("database.db")
            cur = conn.cursor()
            cur.execute("SELECT ID, Location, Status, Flow, Accum, Operator FROM estaciones")
            filas = cur.fetchall()
            for fila in filas:
                estado = fila[2]
                fila_limpia = [valor if valor is not None else "-" for valor in fila]
                fila_limpia[2] = TEXTO_ESTADO.get(estado, estado)

                tag = estado
                if estado == "WATERING" and not parpadeo["encendido"]:
                    tag = "WATERING_OFF"

                tabla.insert("", tk.END, values=fila_limpia, tags=(tag,))
            conn.close()
        except Exception as e:
            print(f"[ERROR BD] No se pudieron refrescar los datos: {e}")

        # Re-ejecutar cada 500 milisegundos (también marca el ritmo del parpadeo)
        ventana.after(500, cargar_datos_bd)

    # 2. Frame Derecho: Imagen del Mapa
    frame_der = tk.Frame(ventana, bg="white", relief=tk.SUNKEN, bd=2)
    frame_der.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True, padx=10, pady=10)

    lbl_mapa = tk.Label(frame_der, text="Mapa de Zonas", font=("Arial", 14, "bold"), bg="white")
    lbl_mapa.pack(pady=5)

    try:
        imagen_mapa = tk.PhotoImage(file="InvernaliaZonas.png")
        lbl_imagen = tk.Label(frame_der, image=imagen_mapa, bg="white")
        lbl_imagen.image = imagen_mapa
        lbl_imagen.pack(expand=True)
    except tk.TclError:
        lbl_error = tk.Label(frame_der, text="[Imagen no encontrada]\nAsegúrate de que 'InvernaliaZonas.pn
g'\nestá en el mismo directorio.", bg="white", fg="red")
```

```
        lbl_error.pack(expand=True)

    # Iniciar la carga de datos y el bucle principal de la interfaz
    cargar_datos_bd()
    ventana.mainloop()

######################### MAIN ##########################

if len(sys.argv) == 2:
    cargar_estaciones_bd()
    print(f"[BD] {len(estaciones)} estaciones cargadas de la base de datos (todas en OFFLINE)")
    mostrar_bd()

    PORT = int(sys.argv[1])
    SERVER = "0.0.0.0"
    ADDR = (SERVER, PORT)

    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind(ADDR)

    print(f"[STARTING] Iniciando WM_Central en {SERVER}:{PORT}...")
    hilo_servidor = threading.Thread(target=start, args=(server, SERVER, PORT), daemon=True)
    hilo_servidor.start()
    Pantalla_Monitor()
else:
    print("Uso: python WM_Central.py <puerto_escucha>")
    print("Ejemplo: python WM_Central.py 9999")

conexion.close()
```

A.2 WM_WS_M.py

```python
import socket
import sys
import threading
import time

HEADER = 64
FORMAT = 'utf-8'
FIN = "FIN"

# Protocolo Monitor <-> Engine
HEALTH = "HEALTH"       # el monitor lo envía cada segundo
OK = "OK"               # respuesta del engine si todo va bien
KO = "KO"               # respuesta del engine si detecta avería (fuga)
TIMEOUT_ENGINE = 3      # segundos sin respuesta del engine = avería

# Estados que el monitor comunica a la central (mismos nombres que en WM_Central)
AVAILABLE = "AVAILABLE"
LEAK = "LEAK"

# El socket con la central lo usan dos hilos (menú y vigilancia del engine).
# El lock garantiza que cada send va seguido de SU receive sin mezclarse.
lock_central = threading.Lock()
estado_ws = AVAILABLE


def send(client, msg):
    message = msg.encode(FORMAT)
    header = f"{len(message):<{HEADER}}".encode(FORMAT)
    client.sendall(header + message)


def recive_msg(sock, size):
    solu = None
    data = bytearray()
    while len(data) < size:
        part = sock.recv(size-len(data))

        if not part:
            return solu
        data.extend(part)

    return bytes(data)

def receive(client):
    solu = None
    header = recive_msg(client, HEADER)
    if header == solu:
        return solu

    msg_length = int(header.decode(FORMAT).strip())
    message = recive_msg(client, msg_length)

    if message == solu:
        return solu

    return message.decode(FORMAT)


def hablar_central(client, msg):
    # Envía un mensaje a la central y devuelve su respuesta (None si se ha caído)
    with lock_central:
        try:
            send(client, msg)
            return receive(client)
        except OSError:
```

```
            return None


def cambiar_estado(client, id_ws, nuevo_estado):
    # Solo se avisa a la central cuando el estado cambia de verdad (transición)
    global estado_ws
    if nuevo_estado != estado_ws:
        estado_ws = nuevo_estado
        print(f"\n[ESTADO] {id_ws} pasa a {nuevo_estado}")
        respuesta = hablar_central(client, f"ESTADO#{id_ws}#{nuevo_estado}")
        if respuesta is None:
            print("[ERROR] No se pudo notificar el estado a la central")
        else:
            print(f"[RESPUESTA CENTRAL]: {respuesta}")


def vigilar_engine(conn_engine, client, id_ws):
    # Cada segundo pregunta al engine por su salud. KO o silencio = fuga.
    conn_engine.settimeout(TIMEOUT_ENGINE)
    conectado = True

    # Lo primero es decirle al engine qué ID tiene (ya validado por la central)
    try:
        send(conn_engine, f"ID#{id_ws}")
    except OSError:
        conectado = False

    while conectado:
        try:
            send(conn_engine, HEALTH)
            respuesta = receive(conn_engine)
        except OSError:  # incluye socket.timeout y conexión reseteada
            respuesta = None

        if respuesta == OK:
            cambiar_estado(client, id_ws, AVAILABLE)
        elif respuesta is None:
            print("\n[ENGINE] El engine no responde o se ha desconectado")
            cambiar_estado(client, id_ws, LEAK)
            conectado = False
        else:
            print(f"\n[ENGINE] Respuesta de avería recibida: {respuesta}")
            cambiar_estado(client, id_ws, LEAK)

        if conectado:
            time.sleep(1)

    conn_engine.close()


def servidor_engine(puerto_engine, client, id_ws):
    # Servidor de sockets que el monitor ofrece a WM_WS_E (un engine a la vez)
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind(("0.0.0.0", puerto_engine))
    server.listen(1)
    print(f"[ENGINE] Esperando conexión del engine en el puerto {puerto_engine}")

    while True:
        conn_engine, addr_engine = server.accept()
        print(f"\n[ENGINE] Engine conectado desde {addr_engine}")
        vigilar_engine(conn_engine, client, id_ws)
        print(f"[ENGINE] Esperando reconexión del engine en el puerto {puerto_engine}")


def main():
    num_argumentos = len(sys.argv)
    solu = None
    solu2 = True
```

```
    if num_argumentos != 6:
        print("Uso: python WM_WS_M.py <puerto_engine> <ip_central> <puerto_central> <id_ws> <ubicacion>")
        print('Ejemplo: python WM_WS_M.py 8000 127.0.0.1 9999 WS_01 "Bosque de los Dioses"')
    else:
        puerto_engine = int(sys.argv[1])
        server_central = sys.argv[2]
        puerto_central = int(sys.argv[3])
        id_ws = sys.argv[4]
        ubicacion = sys.argv[5]
        addr = (server_central, puerto_central)
        client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        try:
            client.connect(addr)
        except OSError:
            print(f"No se pudo conectar con la central en {addr}")
            return
        print(f"Conexión establecida con {addr}")

        msg_registro = f"REGISTER#{id_ws}#{ubicacion}"
        send(client, msg_registro)

        confirmacion = receive(client)

        if confirmacion == solu or not confirmacion.startswith("ACK#REGISTER_OK"):
            print(f"Registro rechazado o conexión cerrada por la central: {confirmacion}")
            client.close()
            return

        print(f"[RESPUESTA CENTRAL]: {confirmacion}")

        # Hilo que atiende al engine y vigila su salud. daemon=True para que
        # muera solo cuando el monitor termine con FIN.
        hilo_engine = threading.Thread(target=servidor_engine, args=(puerto_engine, client, id_ws), daemon=Tr
ue)
        hilo_engine.start()

        while solu2:
            msg = input("Introduce comando/estado o 'FIN' para salir: ").strip()

            if not msg:
                continue

            if msg == FIN:
                with lock_central:
                    send(client, msg)
                print("[DESCONEXIÓN] Cerrando conexión.")
                break

            respuesta = hablar_central(client, msg)
            if respuesta is solu:
                print("[DESCONEXIÓN] La central cerró la conexión")
                break

            print(f"[RESPUESTA CENTRAL]: {respuesta}")

        client.close()


########## MAIN ##########

print("****** WELCOME TO OUR BRILLIANT SD UA CURSO 2026/2027 SOCKET CLIENT ****")
main()
```

A.3 WM_WS_E.py

```python
import socket
import sys
import threading
import time

HEADER = 64
FORMAT = 'utf-8'
FIN = "FIN"

# Protocolo Monitor <-> Engine (el mismo que en WM_WS_M)
HEALTH = "HEALTH"       # pregunta del monitor cada segundo
OK = "OK"               # todo funciona
KO = "KO"               # avería simulada (fuga)
REINTENTO_MONITOR = 3   # segundos entre intentos de conexión al monitor

# Estado de salud que se responde al monitor. Lo cambia el teclado (hilo principal)
# y lo lee el hilo del monitor; es una simple asignación, no hace falta lock.
estado_engine = OK
id_ws = None


def send(client, msg):
    message = msg.encode(FORMAT)
    header = f"{len(message):<{HEADER}}".encode(FORMAT)
    client.sendall(header + message)


def recive_msg(sock, size):
    solu = None
    data = bytearray()
    while len(data) < size:
        part = sock.recv(size-len(data))

        if not part:
            return solu
        data.extend(part)

    return bytes(data)

def receive(client):
    solu = None
    header = recive_msg(client, HEADER)
    if header == solu:
        return solu

    msg_length = int(header.decode(FORMAT).strip())
    message = recive_msg(client, msg_length)

    if message == solu:
        return solu

    return message.decode(FORMAT)


def atender_monitor(addr_monitor):
    # Se conecta al monitor y responde a cada HEALTH con el estado actual.
    # Si el monitor se cae, vuelve a intentarlo cada REINTENTO_MONITOR segundos.
    global id_ws
    while True:
        try:
            conn = socket.create_connection(addr_monitor)
        except OSError:
            print(f"\n[MONITOR] No se puede conectar a {addr_monitor}, reintentando en {REINTENTO_MONITOR}s")
            time.sleep(REINTENTO_MONITOR)
            continue
```

```
66
67          print(f"\n[MONITOR] Conectado al monitor en {addr_monitor}")
68          conectado = True
69          while conectado:
70              try:
71                  mensaje = receive(conn)
72              except OSError:
73                  mensaje = None
74
75              if mensaje is None:
76                  conectado = False
77              elif mensaje.startswith("ID#"):
78                  id_ws = mensaje.split('#')[1]
79                  print(f"[MONITOR] ID asignado a este engine: {id_ws}")
80              elif mensaje == HEALTH:
81                  try:
82                      send(conn, estado_engine)
83                  except OSError:
84                      conectado = False
85              else:
86                  print(f"\n[MONITOR] Mensaje desconocido: {mensaje}")
87
88          conn.close()
89          print(f"\n[MONITOR] Conexión con el monitor perdida, reintentando en {REINTENTO_MONITOR}s")
90          time.sleep(REINTENTO_MONITOR)
91
92
93  def main():
94      global estado_engine
95      num_argumentos = len(sys.argv)
96      if num_argumentos != 5:
97          print("Uso: python WM_WS_E.py <ip_broker> <puerto_broker> <ip_monitor> <puerto_monitor>")
98          print("Ejemplo: python WM_WS_E.py 127.0.0.1 9092 127.0.0.1 8000")
99      else:
100         # Los datos del broker se usarán cuando se añada Kafka
101         ip_broker = sys.argv[1]
102         puerto_broker = int(sys.argv[2])
103         ip_monitor = sys.argv[3]
104         puerto_monitor = int(sys.argv[4])
105         addr_monitor = (ip_monitor, puerto_monitor)
106
107         # Hilo que habla con el monitor. daemon=True para que muera con FIN.
108         hilo_monitor = threading.Thread(target=atender_monitor, args=(addr_monitor,), daemon=True)
109         hilo_monitor.start()
110
111         # El hilo principal se queda leyendo el teclado para simular averías
112         print("Comandos: K = provocar avería (KO) | O = resolver avería (OK) | FIN = salir")
113         salir = False
114         while not salir:
115             tecla = input().strip().upper()
116
117             if tecla == "K":
118                 estado_engine = KO
119                 print("[AVERÍA] Fuga simulada: se responderá KO al monitor")
120             elif tecla == "O":
121                 estado_engine = OK
122                 print("[AVERÍA] Fuga resuelta: se responderá OK al monitor")
123             elif tecla == FIN:
124                 print("[DESCONEXIÓN] Apagando engine.")
125                 salir = True
126             elif tecla:
127                 print("Comando no válido. Usa K, O o FIN")
128
129
130 ########## MAIN ##########
131
132 print("****** WM_WS_E - ENGINE DE ESTACIÓN DE RIEGO ****")
133 main()
```